Measuring Semantic Preservation in Passkey Migration

Anonymous Authors

Abstract

An imported passkey can still authenticate while a property needed by an application has changed. We present Passkey-Transit, a reproducible harness that measures identity, WebAuthn behavior, extension state, and operational guarantees separately. A balanced synthetic campaign exercised 6,144 credential–route–repetition attempts; 5,120 reached import and 1,024 were rejected before a ceremony. Every executed assertion authenticated, yet 512 imported attempts failed the executed browser largeBlob oracle; the user-verification oracle had no failures. These fractions describe the constructed matrix, not product prevalence. Follow-up controls distinguish a project-authored CXP binding from selected provider behavior. A Chromium virtual-authenticator transfer preserved the signing credential but lost its PRF result; four public assertions from a replay were independently reverified. Fenko authenticated after CXF import while losing the original HMAC seeds. A Proton software authenticator preserved both seeds and reproduced the expected PRF with user verification. We provide claim boundaries and public verifiers. We did not establish public CXP wire interoperability between providers or a verified Android transfer.

1 Introduction

An accepted import and successful login are weak tests of passkey migration. A relying party can also depend on the credential identifier, WebAuthn extension output, user-verification behavior, and whether transfer retries leave duplicate state. The Credential Exchange Format (CXF) represents credentials [1]; the draft Credential Exchange Protocol (CXP) addresses protected exchange [2]. WebAuthn authenticates an assertion [5], but a valid signature does not by itself show that every credential property survived the move.

We model a credential as $C = (I, K, F, S)$: identity, signing material, observable functions, and operational guarantees. Our falsifiable proposition is that valid authentication does not imply preservation of every component. We contribute: (1) a claim matrix and property-specific oracles; (2) a synthetic campaign with explicit denominators and non-evaluable properties; (3) executable controls using two project-authored runtimes, Chromium virtual authenticators, and selected open-source provider code; and (4) an artifact whose public reports allow independent checks of retained signatures and PRF digest relationships. This is a measurement method with selected software cases, not a survey of deployed products.

2 Background and related work

CXF can encode FIDO2 HMAC state [1]. The CXP working draft defines a transport profile [2]; our single-document CXP/HPKE binding is an experimental project implementation and does not cover its full ZIP/JWE wire format [4]. HPKE follows RFC 9180 [3]. A WebAuthn PRF result is returned through client extension results, outside the ordinary signed assertion bytes [5]. Consequently, signature and PRF oracles answer separate questions. Ramat et al. study experiences across 111 websites [6]; Jannett et al. measure deployment [7]; Büttner and Gruschka compare device-bound and synced credentials [8]. Daffalla et al. study interpersonal abuse [9]; Islam et al. study cloud storage compromise [10]; Henarejos proposes authenticated device-bound restoration [11]. These studies address relying-party experience, deployment, storage, abuse, or a proposed recovery architecture. Our unit is a controlled migration attempt with a property-specific post-import oracle; we do not measure live-site security or prove a transfer protocol secure.

Our unit is an attempted migration of a controlled credential, with separate post-import oracles for signing, extension behavior, and retry state. The method makes loss of observability an explicit outcome, rather than interpreting a successful assertion as evidence about unexecuted properties. The controlled routes expose that distinction, while the selected provider cases test only a few concrete code paths. We do not infer website adoption, attack prevalence, or provider prevalence.

3 Study design

3.1 Units, outcomes, and exclusions

The v1.5 protocol crossed 256 synthetic ES256 credentials (32 in each of eight strata) with 12 routes and two repetitions: 6,144 attempts in 96 route-by-stratum groups. The strata covered basic credentials, PRF with and without UV, largeBlob, credBlob, a payment-format marker, combined extensions, and a future optional member. They were constructed controls, not random draws. A route could reject the document before WebAuthn. For each applicable property we record PASS, FAIL, NOT_APPLICABLE, or NOT_EVALUABLE. The last outcome is needed where CDP cannot inject PRF or credBlob state.

One oracle checks CXF representation and policy. Each imported attempt performs two WebAuthn assertions: the first binds a challenge to the attempt, credential, route, repetition, public key, user handle, RP ID, and origin; the second binds its challenge to the first and to a canonical digest of extension observations. A separate verifier checks both ES256 signatures, the RP hash, user-presence and verification flags, and counter. Extension oracles examine executed behavior where the interface permits it; representation loss alone is never relabeled as an executed PRF failure. A deterministic in-memory simulation tests transfer atomicity and retry behavior. These oracles are not interchangeable. Campaign percentages are design-weighted proportions, without population confidence intervals; paired repetitions are stability checks, not independent providers. The browser and provider follow-ups were designed after inspecting earlier results and are exploratory rather than preregistered confirmations.

3.2 Estimand and evidentiary levels

Let A_i mean that attempt i imported and its original-RP assertion passed. Let B_i mean that an applicable, executed UV or largeBlob oracle failed. Our primary constructed-matrix estimand is $\sum_i A_i B_i / \sum_i A_i$, the share of successful imported assertions that coexist with an observed browser behavior failure. We do not use a missing PRF result when the browser cannot restore the original HMAC seed to set B_i: that property is NOT_EVALUABLE. Missing serialized PRF or credBlob state is a separate representation failure. A payment marker is format evidence, not a payment ceremony. Thus PASS means all applicable *observable* oracles passed; it is not proof that every latent credential property survived.

The 256 credentials are reused across routes, and each credential–route cell runs twice. The 96 route–stratum cells are a balanced census of chosen inputs and policies, not independently sampled people, sites, or providers. We report exact design-weighted fractions and route contrasts, without population confidence intervals. Earlier C1 runs were inspected before corrective changes to strict rejection, challenge binding, and extension-witness handling. Protocol v1.5 and its replay supersede those runs; the final census is not a preregistered confirmation.

Table 1: C1 synthetic attempts; denominator 6,144.

Classification	Count	Share
Complete observable PASS	2,304	37.50%
Visible degradation	960	15.62%
Silent degradation	1,344	21.88%
Not evaluable	512	8.33%
Pre-ceremony rejection	1,024	16.67%

3.3 Threat and implementation boundary

Two separately written project implementations exchange one CXF document in Python-to-Node and Node-to-Python directions and reject a modified ciphertext. This tests that experimental binding. Exporter identity, authorization, durable provider storage, and the public CXP ZIP/JWE envelope are outside the tested profile. The transaction model is in memory and makes no claim about crash durability.

The reference route models an importer that retains supported fields. The strict route refuses a credential when it cannot preserve a required property. The compatible-lossy route accepts a credential while dropping selected extension state; the legacy route has a different loss policy. These are controlled policies written for this study, not named vendors. Direct, round-trip, and multihop chains expose whether a later capable destination can recover a field already lost at an earlier hop.

The source credential, expected CXF fields, and route policy are known to the harness. Its representation oracle therefore has ground truth that a real RP usually lacks. The browser oracle sees only what a WebAuthn ceremony and client extension results expose. This separation prevents a missing serialized seed from being counted as a failed browser PRF evaluation. The same distinction applies to a payment-format marker without a Secure Payment Confirmation ceremony.

4 Synthetic campaign results

Of 6,144 attempts, 1,024 were rejected before ceremony and 5,120 were imported. Every executed assertion among the imported attempts was accepted. Table 1 separates those accepted assertions from other oracles. The complete observable PASS yield was 2,304/6,144 (37.50%). This describes the balanced test design, not the probability that an arbitrary user or product will succeed.

Among imported attempts, 512 (10.00%) combined valid login with failure of the executed largeBlob browser oracle; UV had no failures. PRF or credBlob representation loss

Table 2: C1 outcomes by route. Each route has 512 designed attempts. P: complete observable PASS; V: visible degradation; S: silent degradation; NE: not evaluable. Counts are exact for this design.

Route	Chain	Imported	Rejected	P	V	S	NE
R01	ref–strict	512	0	256	0	0	256
R02	ref–lossy	512	0	256	256	0	0
R03	ref–legacy	512	0	128	0	384	0
R04	ref–strict–ref	512	0	256	0	0	256
R05	ref–lossy–ref	512	0	256	256	0	0
R06	ref–legacy–ref	512	0	128	0	384	0
R07	ref–strict–lossy	512	0	256	256	0	0
R08	ref–lossy–strict	256	256	256	0	0	0
R09	ref–strict–legacy	512	0	128	0	384	0
R10	ref–legacy–strict	128	384	128	0	0	0
R11	ref–lossy–legacy–strict	128	384	128	0	0	0
R12	ref–strict–lossy–legacy–ref	512	0	128	192	192	0
Total		5,120	1,024	2,304	960	1,344	512

Table 3: C1 outcomes by constructed credential stratum. Each stratum has 768 designed attempts over all routes and repetitions. Labels F0–F7 are defined in the text.

Stratum	Imported	Rejected	P	V	S	NE
F0 basic	768	0	768	0	0	0
F1 PRF with UV	576	192	0	256	192	128
F2 PRF without UV	576	192	0	256	192	128
F3 largeBlob	640	128	384	0	256	0
F4 credBlob	576	192	0	256	192	128
F5 payment marker	640	128	384	0	256	0
F6 combined extensions	576	192	0	192	256	128
F7 optional member	768	0	768	0	0	0
Total	5,120	1,024	2,304	960	1,344	512

occurred in 1,792 (35.00%); this category overlaps the former and does not establish an executed PRF failure in those cells. The union of non-payment failures was 2,048/5,120 (40.00%). A payment-format marker raises the design sensitivity to 2,304/5,120 (45.00%), but is not a Secure Payment Confirmation ceremony. C2 malformed document controls comprised 80 cases across ten families and eight strata; 64 were rejected by structural validation, duplicate policy, or strict preservation. C3 exercised 960 in-memory failure sequences and 1,920 events: 832 rolled back and converged to one copy on retry, while the 128 deliberate legacy late-failure controls retained a provisional and duplicated on retry. Neither C2 nor C3 is included in the C1 percentages.

Table 2 shows the route dependence directly. R03, R06, and R09 each contain 384 silent-degradation attempts, despite 512 imports and valid assertions. In the paired R02/R05 comparison, returning to the reference provider leaves the same 256 visible-degradation attempts: a later capable endpoint cannot reconstruct dropped source state. Strict handling after a lossy hop rejects 256 attempts in R08; after a legacy hop it rejects 384 in R10. These are consequences of the specified control policies, not failure rates for real providers.

Table 3 separates which controls generate the aggregate. The basic and optional-member strata each have 768 observable PASS attempts. PRF-bearing and credBlob strata have no complete PASS because their positive browser behavior cannot be established through the CDP injection path even when their serialized representation is retained. The 512 false-reassurance cases occur in largeBlob-bearing strata F3 and F6: 256 in each, with accepted assertions and a failing executed browser behavior oracle. These 512 cases are a subset of the broader semantic-degradation counts, not an additional category to add to Table 1.

The C2 family outcome matters more than its pooled rejection count. Eight unknown-optional-member and eight minor-version-increase cases were imported but assigned NOT_EVALUABLE; a credential-ID collision was rejected in eight cases while its normative class remained ambiguous.

C3's 128 failed atomicity and idempotence controls were exactly the two late failure points of the legacy model, 64 each. Because C3 has no process crash or durable storage, its result cannot predict recovery behavior of a password manager.

5 Executable follow-up controls

5.1 Experimental exchange and browser PRF loss

Python and Node recovered identical CXF documents in both directions and rejected ciphertext alteration. Software PRF evaluators agreed on retained CXF HMAC seeds and detected their removal. This is a functional software control, not a browser ceremony.

A Chromium virtual authenticator created a PRF-enabled passkey. Three source assertions showed a PRF result stable for one salt and different for a second. The runner exported the signing credential via CDP, carried the CXF document through the experimental exchange, and injected the credential into a new virtual authenticator. A fourth assertion authenticated at the destination, while the PRF result for the original salt was absent. CDP did not expose a PRF seed. This is a loss control for that test path, not a finding about a commercial credential provider.

A Windows 2025 GitHub Actions replay recorded six exchange and nine browser checks as true. A second replay retained four public signatures, the registration public key, challenges, authenticator and client data, and SHA-256 digests of observed PRF outputs. A separate verifier checked four ES256 signatures, RP binding, challenge uniqueness, UV flags, credential identity, and four PRF digest relationships. The archive contains no private key, seed, or raw PRF output. Comparing hashes proves relations among observed outputs, not the HMAC computation from a seed that CDP did not reveal.

5.2 Public-draft envelope structure probe

Our synthetic CXP draft shape probe passed visible request/response, offer-selection, and index/document JWE ZIP checks. It rejected three malformed responses, unexpected ZIP paths, and JSON substituted for JWE. Its direct-key AES-GCM fixture tests container shape only. The displayed draft objects omit narrative signed-challenge fields and a complete HPKE-to-JWE binding. Negotiated HPKE authentication, challenge checks, CXF recovery, and provider exchange are NOT_EVALUABLE in this parsing probe [2].

Pinned Bitwarden Rust [12] and `go-cxp` Go [13] types both parsed a synthetic request/response in CI. Rust serialization dropped request `archive`/`mode` and response `archive`; Go dropped request `mode`. Both rejected 256 with eight-bit version types, despite the draft's wider field. These are parser gaps, not encrypted transfers.

Project Python and Node recovered synthetic CXF from a private two-member HPKE/JWE ZIP (nine local and four cross-runtime controls). Independent Go `jwx/v4` exchanged `HPKE-3-KE` JWE with Python in both directions (five controls), then decrypted our two-member CXP-shaped ZIP (five controls) [18, 19]. Both ZIP bindings remain project-authored. Proton Pass common 2.0.0 [17] and Python exchanged direct-HPKE responses in both directions. Credential ID, key, and HMAC seeds survived; four software assertions and four PRF checks passed on the Proton-to-Python path. Proton's direct payload lacks the draft's required ZIP/JWE structure [2], and no provider vault import or deployed RP assertion was tested.

The open `cxporter` exporter [14] produced index and document JWEs for two synthetic passkeys. Our Python importer decrypted them, checked identity, key, user handle and HMAC state, and rejected wrong recipients and altered ciphertext. Proton software then yielded eight verified signatures and PRF results. This external exporter to project importer test is not two-provider exchange: its response omits the draft-required `archive` and uses a different JWE algorithm than our JOSE control [2]. An open research importer passed its own passkey transfer but rejected this export's JWE algorithm before CXF recovery [15]. A retained synthetic ciphertext vector permits replay of the same response: its full schema parse rejects a missing `archive`; adding only that field passes schema parsing, after which the importer rejects the JWE algorithm. This isolates two gates without implying draft nonconformance. In a separate transformed control, we re-encrypted the decrypted `cxporter` index inside the second importer's own envelope. Its self-import passed, but the substituted index was rejected for lacking `header` and `documents`. This locates a later layout boundary; the transformed ZIP is not the original export. Separately, the PoC's own exporter and importer transferred one synthetic passkey in four direct/indirect and policy combinations; its local RP verified login before and after import with one credential ID. In two further runs, a project decryptor opened fresh `cxporter` ZIPs, inserted recovered CXF keys directly into the PoC vault, and the RP verified them. These software controls do not establish provider interoperability: the adapter bypasses the PoC's CXP importer, which rejects the original response.

5.3 Selected provider case: Fenko

We selected Fenko Vault 0.12.0 [16] after inspecting its CXF path. Two fresh profiles per repetition imported synthetic basic and PRF-bearing credentials. The credentials signed valid assertions. A public verifier rechecked 16 signatures and six PRF observation bindings. The source CXF HMAC seeds were not retained in the observed import/export path; the browser PRF result differed from an oracle derived from those source seeds.

A same-provider round trip between two Fenko profiles produced matching PRF outputs in two selected repetitions. A

separate verifier recalculated 24 signatures and 12 PRF bindings. This continuity does not imply that the original seeds survived: this code path derived a value from the imported signing key. Functional equality and preservation of the represented secret are separate claims. Two repetitions are not a product-level reliability estimate.

5.4 Positive software control: Proton

In two exploratory repetitions, the Proton Pass common 2.0.0 software authenticator [17] imported, exported and reimported CXF. Both HMAC seeds survived; with UV, the expected PRF recurred for two salts. Changing the seed changed PRF while signing still worked. The public verifier checked 12 signatures and PRF digests, including four changed-seed controls. Computing HMAC from the withheld seed requires replay. This pinned WASM route used neither an installed provider app nor a no-UV ceremony.

5.5 Comparison of the selected execution paths

These follow-ups cross different boundaries. The virtual-authenticator control crosses a browser serialization path but cannot inject the original PRF seed. Fenko runs an installed extension in Chromium and produces signed assertions, while the Proton control exercises pinned software authenticator code without a browser or installed vault app. The `cxporter` ZIP is generated by external code and decrypted by our importer; a second open research importer rejects its response schema and JWE algorithm before any CXF recovery. Passing one boundary does not establish a stronger one: a parsed response is not a decrypted payload, a decrypted payload is not a durable vault import, and a valid assertion does not certify extension state outside the signed WebAuthn data.

6 Discussion and limitations

The browser loss control and Fenko case show different ways a signing credential can outlive its original PRF state. The Proton software route shows that preserving CXF seeds and reproducing the expected PRF is possible in one selected implementation. These cases are not a representative sample of provider behavior.

The public CXP wire format was not exchanged between two independent providers. Android's provider-events flow carries CXF JSON through a content URI [20], so a mobile transfer would test a different path. The human-reported Bitwarden-to-Google transfer lacks an instrumented destination assertion and receipt and is excluded from measured results. CDP cannot inject original PRF state; positive browser-to-browser restoration of CXF HMAC seeds is NOT_EVALUABLE. The Proton module result cannot fill that gap. The transaction simulation cannot support claims about storage durability, and the synthetic campaign cannot support product prevalence or superiority claims.

6.1 Threats to validity

Construct validity. The controlled route policies deliberately retain, reject, or discard properties. Their resulting proportions test whether the oracles distinguish those outcomes; they do not estimate how often real products lose passkey state. An RP accepts an assertion based on the signed challenge and authenticator data, while PRF is returned as a client extension result. Our public signature verifiers and PRF digest oracles therefore establish different properties. A matching output digest can bind two observations, but cannot reveal a withheld HMAC seed.

Internal and external validity. We revised the protocol after seeing earlier results and treat later provider cases as selected exploratory paths. Two repetitions expose deterministic mistakes but do not measure intermittent failure rates. Virtual authenticators, pinned open libraries, and research exporters differ from mobile provider apps, account synchronization, and durable storage. The negative CXP pairing identifies concrete profile differences; draft silence about the HPKE–JWE binding and index contents prevents a conformance verdict.

Reproducibility. Public reports permit recalculation of retained signatures and some synthetic PRF relations. ZIP decryption, parser behavior, and software authenticator state require execution of pinned source. Clean CI replays were run by the same team; they are environment replications, not independent experiments. The staged anonymous archive has no reviewer URL yet, and no external audit is claimed.

7 Conclusion

PasskeyTransit uses distinct oracles for identity, signing, extensions, and transaction semantics, marking properties an interface cannot expose. The synthetic campaign and selected controls show valid authentication alongside changed or missing properties. Public replays verify observed signatures; provider interoperability and mobile transfer remain open.

A Open Science

Staged source, synthetic rows, a public ciphertext vector, reports and original CI ZIPs include verifiers for 96 C1 cells, 52 provider signatures and 30 PRF bindings. Clean CI is same-team execution, without independent replication or provider-app CXP transfer. Vaults and secret material are excluded; run mapping is separate and an anonymous URL is pending.

B Ethical considerations

All credentials and RP accounts were synthetic; browser controls used virtual authenticators and localhost. No real passkey or vault is distributed. Android remains unverified; provider findings apply only to tested routes and revisions.

References

[1] FIDO Alliance. Credential Exchange Format v1.0 Proposed Standard Errata, 2026. https://fidoalliance.org/specs/cx/cxf-v1.0-ps-errata-20260309.html.

[2] FIDO Alliance. Credential Exchange Protocol v1.0 Working Draft, 3 October 2024. https://fidoalliance.org/specs/cx/cxp-v1.0-wd-20241003.html.

[3] R. Barnes et al. Hybrid Public Key Encryption. RFC 9180, 2022. https://www.rfc-editor.org/rfc/rfc9180.

[4] M. Jones and J. Hildebrand. JSON Web Encryption. RFC 7516, 2015. https://www.rfc-editor.org/rfc/rfc7516.

[5] W3C. Web Authentication Level 3, Recommendation, 25 August 2026. https://www.w3.org/TR/2026/REC-webauthn-3-20260825/.

[6] B. Ramat, D. Kartchner, M. Clark, and K. Seamons. Passkeys in the Wild: A Systematic Study of FIDO2 User Experience Consistency Across Websites. SOUPS, pp. 21–39, 2026. https://www.usenix.org/conference/soups2026/presentation/ramat.

[7] L. Jannett, A. Mayer, M. Westers, V. Mladenov, C. Mainka, and J. Schwenk. The State of Passkeys: Studying the Adoption and Security of Passkeys on the Web. USENIX Security, pp. 5039–5058, 2026. https://www.usenix.org/conference/usenixsecurity26/presentation/jannett.

[8] A. Büttner and N. Gruschka. Device-Bound vs. Synced Credentials: A Comparative Evaluation of Passkey Authentication. ICISSP, 2025. https://arxiv.org/abs/2501.07380.

[9] A. Daffalla et al. A Framework for Abusability Analysis: The Case of Passkeys in Interpersonal Threat Models. USENIX Security, pp. 7819–7838, 2025. https://www.usenix.org/conference/usenixsecurity25/presentation/daffalla.

[10] M. Islam et al. Detecting Compromise of Passkey Storage on the Cloud. USENIX Security, pp. 7743–7762, 2025. https://www.usenix.org/conference/usenixsecurity25/presentation/islam.

[11] P. Henarejos. Vaulted Passkeys: A Device-Bound Proposal for Authenticated Credential Export and Import. arXiv:2608.13806, 2026. https://arxiv.org/abs/2608.13806.

[12] Bitwarden. Credential Exchange Protocol Rust types, revision `32da4c8`, 2026. https://github.com/bitwarden/credential-exchange.

[13] N. Vinuesa. `go-cxp`, revision `94659f3`, 2026. https://github.com/nvinuesa/go-cxp.

[14] N. Vinuesa. `cxporter`, revision `96bab31`, 2026. https://github.com/nvinuesa/cxporter.

[15] ShaminAkarsha. `cxp_poc2`, revision `fee19d9`, 2026. https://github.com/ShaminAkarsha/cxp_poc2.

[16] FenkoHQ. Passkey Vault 0.12.0, revision `2d873fb`. https://github.com/FenkoHQ/passkey-vault.

[17] Proton. Proton Pass common 2.0.0; pinned revisions in artifact. https://github.com/protonpass/proton-pass-common.

[18] T. Reddy et al. Use of HPKE with JWE, Internet-Draft `draft-ietf-jose-hpke-encrypt-22`, 2026. https://datatracker.ietf.org/doc/draft-ietf-jose-hpke-encrypt/22/.

[19] lestrrat-go. `jwx/v4`, v4.6.0, 2026. https://github.com/lestrrat-go/jwx/tree/v4.6.0.

[20] Android Developers. Credential transfer, 2026. https://developer.android.com/identity/sign-in/credential-transfer.